Übungsaufgaben GINF

1. Wählen Sie zufällig eine Zahl zwischen 32 und 64. Geben Sie diese Zahl in Hex-Form einschließlich Rechenweg an!

Z.B.: 55d (d = dezimal)

55 / 16 = 3 Rest 7
3 / 16 = 0 Rest 3

Man lese von unten nach oben und schreibe von links nach rechts: 37h (h = hexadezimal)

2.

a) Programmieren Sie „Sortieren durch Einfügen“ in C für Integerzahlen – Arrays.

```
public void insertion_sort (long array[], int count){
        int i, j;
        long tmp;
        long zstVorher;
        long zstNachher;
        for (i = 1; i < count; i++){
                tmp = array[i];
                for (j = i - 1; j >= 0 && tmp < array[j]; j--){
                        array[j + 1] = array[j];
                }
                array[j+1] = tmp;
        }
}
```

b) Testen Sie das Programm mit sorterten, zufälligen und umgekehrt sortierten Arrays auf Korrektheit.

N	Best Case	Worst Case	Average Case
500000	10	227966	46453
300000	8	69421	16684
100000	6	10641	4466

c) Sie für sortierte, zufällige und umgekehrt sortierte Arrays die Parameter a, b und c von $T(n)=a*n^2+b*n+c$. Da in der Sprache C die Länge statischer Arrays auf 250000 beschränkt ist, sollten Sie mit dynamischen Arrays arbeiten- Z.B.:

```
int* a;                          /* Verweis auf ein Array */
n = 300000000;                   /* Größe des Arrays */
a = (int*) malloc(sizeof(int)*n); /* Speicherplatz reservieren */
                                 /* Initialisierung, sortieren, Ausgabe */
free(a);                         /* Speicherplatz freigeben */
```

$$T(n)=a*n^2+b*n+c$$

Best Case:

$$\begin{aligned} 10 &= a*500000^2+b*500000+c \\ 8 &= a*300000^2+b*300000+c \\ 6 &= a*100000^2+b*100000+c \end{aligned}$$

$$Lösung\ durch\ GTR: a=0,\quad b=1*10^{-5,}\quad c=5$$

Worst Case:

$$\begin{aligned} 227966 &= a*500000^2+b*500000+c \\ 69421 &= a*300000^2+b*300000+c \\ 10641 &= a*100000^2+b*100000+c \end{aligned}$$

$$Lösung\ durch\ GTR: a=1{,}247*10^{-6,}\quad b=-0{,}205,\quad c=18663$$

Average Case:

$$\begin{aligned} 46453 &= a*500000^2+b*500000+c \\ 16684 &= a*300000^2+b*300000+c \\ 4466 &= a*100000^2+b*100000+c \end{aligned}$$

$$Lösung\ durch\ GTR: a=2{,}194*10^{-7,}\quad b=-0{,}027,\quad c=4939$$

d) Bestimmen Sie die theoretische Rechenzeit für ein Array der Länge 5 • 10^9.

$$\begin{array}{rcrcrcrcrcr} Best\ Case & : & T(n) & = & 0*(5*10^9)^2 & + & 1*10^{-5}*5*10^9 & + & 5 & = & 49995 \\ Worst\ Case & : & T(n) & = & 1{,}247*10^{-6}*(5*10^9)^2 & - & 0{,}205*5*10^9 & + & 18663 & = & 3{,}1*10^{13} \\ Average\ Case & : & T(n) & = & 2{,}194*10^{-7}*(5*10^9)^2 & - & 0{,}027*5*10^9 & + & 4939 & = & 5{,}5*10^{12} \end{array}$$

e) Warum wird in der Praxis T(5 • 10^9) viel größer sein als oben errechneter Wert?

Speichergrenzen RAM

f) Durch welche Maßnahme kann man die berechnete Rechenzeit tatsächlich erreichen?

Speicher erweitern

3. Ordnen Sie folgende Funktionen nach Ihrem asymptotischen Wachstum (log ist der Zehnerlogarithmus, ln der natürliche Logarithmus):

$(\ln n)^{\ln n}$	$n^{\frac{3}{2}}$	$n \log n$	$(n+1)!$	$n2^n$	$\ln n$	$\sqrt{n}$	$n!$	$\sqrt{\log n}$
2^n	$\ln \ln n$	n^n	n	$(\ln n!)$	$n^{\frac{2}{3}}$	$\left(\frac{3}{2}\right)^n$	$\log n$	$\ln 2^n$

$\ln \ln n$	$\sqrt{\log n}$	$\log n$	$\ln n$	$\sqrt{n}$	$n^{\frac{2}{3}}$	$\ln 2^n$	n	$n \log n$
$(\ln n!)$	$n^{\frac{3}{2}}$	$(\ln n)^{\ln n}$	$\left(\frac{3}{2}\right)^n$	2^n	$n2^n$	$n!$	$(n+1)!$	n^n

4. Kreuzen Sie in folgender Tabelle alle zutreffenden Felder an. Es seien k ≥ 1, ϵ > 0 und c > 1. Es stehen die Abkürzungen O, o, Ω, ω und Θ für f(n) = O(g(n)), etc. Vergleichen Sie hierzu das asymptotische Verhalten der Funktionen f und g. Zeichnen Sie ggf. die Graphen der beiden Funktionen.

$f(n)$	$g(n)$	O	o	Ω	ω	Θ
n^k	c^n	X	X			
$\log n$	n^ϵ	X	X			
2^n	$2^{n/2}$			X	X	
$n^{\log m}$	$m^{\log n}$	X		X		X
$n!$	n^n	X	X			
$\sqrt{n}$	$n^{\sin n}$					

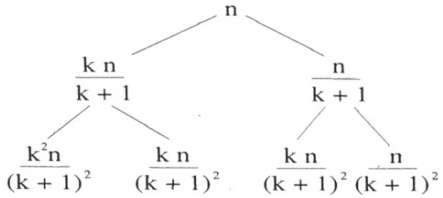
5. Beweisen Sie Satz 3.1, d.h.

$$T(n)=\Theta(g(n)) \quad \Leftrightarrow \quad T(n)=\Omega(g(n)) \text{ und } T(n)=O(g(n)).$$

$$T(n) \leq O(g(n))$$
$$T(n) \geq \Omega(g(n))$$
$$T(n) = \Theta(g(n))$$

Da $O(g(n))$ und $\Omega(g(n))$ jeweils auch $=T(n)$
gilt für $T(n)=\Theta(g(n))$ auch immer $T(n)=\Omega(g(n))$ und $O(g(n))$

6. Skizzieren Sie den Rekursionsbaum von Quicksort für ein konstantes Aufteilungsverhältnis von 1:k und leiten Sie aus diesem Baum $T(n)=O(n\,lg\,n)$ ab.



Tiefe der untersten Ebene: $\left(\frac{k}{k+1}\right)^{d_l} * n = 1$

Rechenaufwand pro Ebene ≤ n

$$d_l * \log\frac{k}{k+1} + \log n = 0 \qquad d_l = \frac{\log n}{\log\frac{k+1}{k}}$$

wegen $T(n) \le n * d_l$ folgt $T(n) = O(n * \log n)$

7. Programmieren Sie Quicksort für einfache Arrays von Integer-Zahlen, indem Sie die Funktion PARTITION wie in der Vorlesung beschrieben implementiere. Zeigen Sie empirisch, dass für zufällig sortierte Arrays $T_{rand}(n)=\Theta(n\,lg\,n)$ und für vorsortierte Arrays $T_{MAX}(n)=\Theta(n^2)$ gilt.

Beispielmessungen:

n	T(n) [sec]
2000000	1.1
4000000	2.47
6000000	3.5
8000000	4.76
10000000	5.84
15000000	9.41
20000000	12.72
40000000	26.98
80000000	55.46

Für $n \to \infty$ gilt $T(n) = Cn\ln n$, also $C = T(n)/n\ln n$
Man erhält also die folgende Tabelle und schließt daraus $C \approx 3.810^{-8}$ sec, d.h. $T(n) \approx 3.810^{-8} n \ln n sec.$

n	$T(n)/(n\ln n)$ [sec]
2000000	$3.79084\ 10^{-8}$
4000000	$4.06202\ 10^{-8}$
6000000	$3.73757\ 10^{-8}$
8000000	$3.74333\ 10^{-8}$
10000000	$3.62326\ 10^{-8}$
15000000	$3.7966\ 10^{-8}$
20000000	$3.78318\ 10^{-8}$
40000000	$3.85332\ 10^{-8}$
80000000	$3.80958\ 10^{-8}$

8. Beweisen Sie, dass für Sortieralgorithmen $T_{min}(n)=\Omega(n)$ gilt.

Weil der Algorithmus auf jedes Element mindestens einmal zugreifen muss, um zu überprüfen, ob es an der richtigen Stelle steht. Ansonsten ist nicht garantiert, dass die gesamte Liste sortiert ist.

9. Gegeben sei folgender Programmteil eines C-Programms:

```
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j++)
        for (k = 1; k <= j; k++)
            z = z + 1;
```

a) Berechnen Sie die Laufzeit *T(n)* und geben Sie die (asymptotische) Zeitkomplexität an.

$$T(n)=n*\sum_{j=1}^{n} j=n*\frac{n(n+1)}{2}=\frac{n}{2}(n^2+n)=\frac{1}{2}(n^3+n^2)=\Theta(n^3)$$

Die äußere Schleife läuft *n* mal durch, daher das erste *n**. Die 2. Schleife läuft auch *n* mal durch und die innere läuft dabei jedes mal *j* mal durch wodurch es passiert, dass beim ersten durchgang davon 1, beim zweiten 2, beim tritten 3 usw. dazugezählt wird, daher kommt diese Summe.

b) Welchen Wert hat *z* nach verlassen der äußeren Schleife für *n = 100*?

Nun kann man das *n* natürlich einfach in die Formel einsetzen:

$$\frac{1}{2}(100^3+100^2) \quad = \quad \frac{1}{2}(1.01*10^6) \quad = \quad 0.505*10^6 \quad = \quad 505000$$

Warum auch immer.. ;)

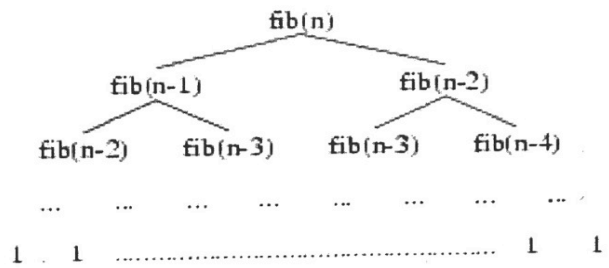
10. Gegeben sei die rekursive Funktion *fib*:

$$\begin{aligned} fib(0) &= 1 \\ fib(1) &= 1 \\ fib(n) &= fib(n-1)+fib(n-2) \quad \text{für } n \geq 2. \end{aligned}$$

a) Berechnen Sie *fib(5)*.

fib(5) = 8

b) Zeichnen Sie den Rekursionsbaum für *fib.*



c) Bestimmen Sie anhand des Rekursionsbaumes die Komplexität von *fib.*

Tiefe des Baumes: n, Zahl der Blätter: 2^n

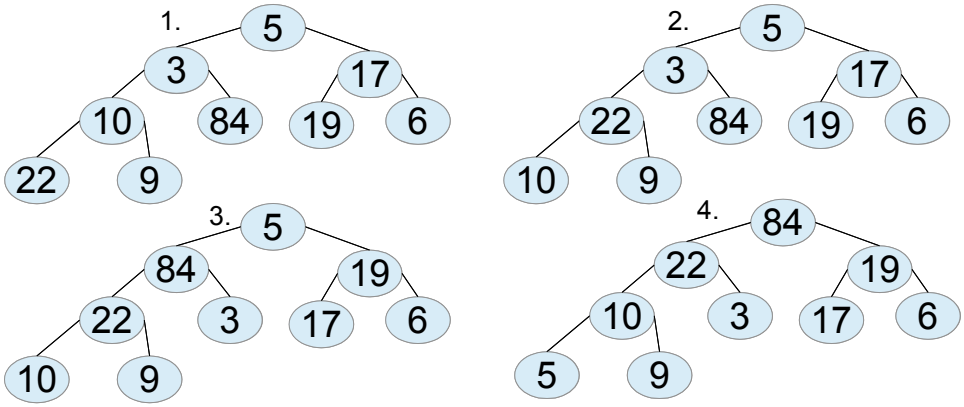
Zahl der Knoten auf Ebene k: 2^k

Zahl der Knoten gesamt: $\sum_{k=0}^{n-1} 2^k = 2^n - 1$

11. Warum wird der Schleifenindex *i* in Zeile 2 von BUILD-HEAP von $\lfloor length[A]/2 \rfloor$ bis 1 erniedrigt und nicht vin 1 bis $\lfloor length[A]/2 \rfloor$ erhöht?

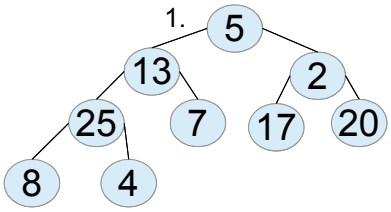
Heapify setzt voraus, dass die Unterbäume schon Heaps sind. Darum muss der Baum von unten nach oben in einen Heap umgewandelt werden.

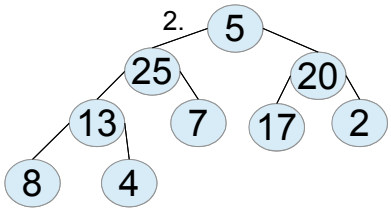
12. Stellen Sie den Ablauf von BUILD-HEAP für A = (5, 3, 17, 10, 84, 19, 6, 22, 9) grafisch dar, ähnlich wie in Beispiel 3.9.

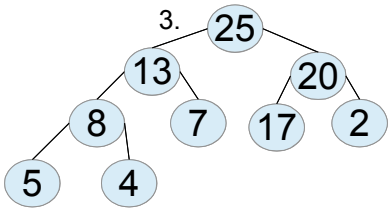


13. Stellen Sie den Ablauf von HEAPSORT für A = (5, 13, 2, 25, 7, 17, 20, 8, 4) dar, ähnlich wie in Beispiel 3.19.

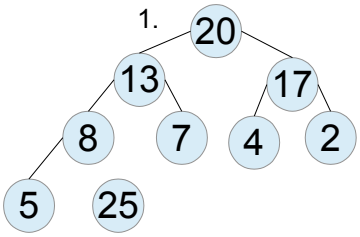
Wende erst den BUILT-HEAP darauf an:

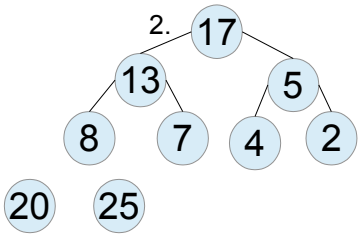


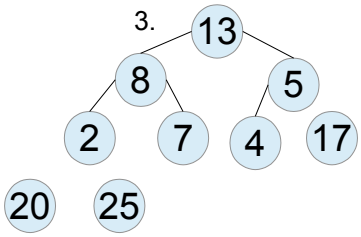


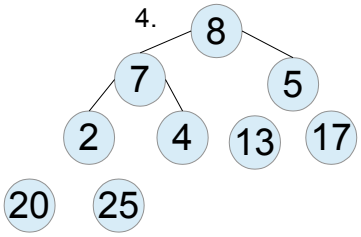


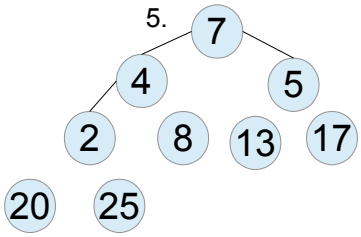
Fahre nun mit dem HEAPSORT fort:

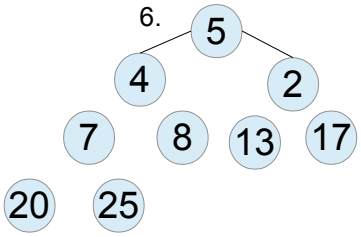


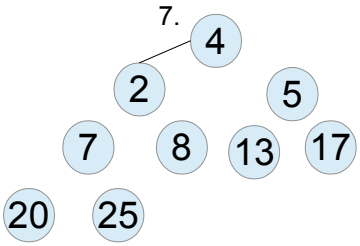


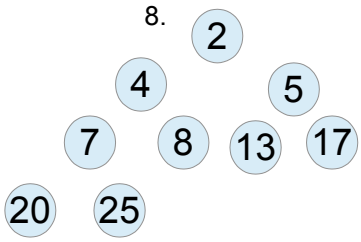












14. Benutzen Sie die Master-Methode zur Berechnung der Laufzeit der Suche durch Bisektion mit der Rekurrenzgleichung $T(n)=T(n/2)+\Theta(1)$.

Aus der Lösung:

$$f(n) = \ln^{\log_2 1} = n^0 = 1 \;\rightarrow\; Fall\,2:\; T(n) = \Theta(n^0 * \log n) = \Theta(\log n)$$

Von mir:

$$\begin{aligned}
T(n) &= T(\tfrac{n}{2})+\Theta(1) \\
&= 1*T(\tfrac{n}{2})+c \quad ;\; a=1,\; b=2,\; f(n)=\Theta(1) \\
n^{\log_b a} &= n^{\log_2 1} = n^0 = 1 \\
f(n) &= \Theta(1) \\
\rightarrow T(n) &= \Theta(n^{\log_b a}\log n) = \Theta(1*\log n) = \underline{\underline{\Theta(\log n)}}
\end{aligned}$$

15. Geben Sie mit Hilfe des Master-Theorems asymptotische Schranken für folgende Rekurrenzen an:

a) $T(n)=4T(n/2)+n$

$$\begin{aligned}
T(n) &= 4T(\tfrac{n}{2})+n \quad ;\; a=4,\; b=2,\; f(n)=n \\
n^{\log_b a} &= n^{\log_2 4} = n^2 \\
f(n) &< n^{\log_b a} \rightarrow Fall\,1 \\
\rightarrow T(n) &= \Theta(n^{\log_b a}) = \underline{\underline{\Theta(n^2)}}
\end{aligned}$$

b) $T(n)=4T(n/2)+n^2$

$$\begin{aligned}
T(n) &= 4T(\tfrac{n}{2})+n^2 \quad ;\; a=4,\; b=2,\; f(n)=n^2 \\
n^{\log_b a} &= n^{\log_2 4} = n^2 \\
f(n) &= n^{\log_b a} \rightarrow Fall\,2 \\
\rightarrow T(n) &= \Theta(n^{\log_b a}\log n) = \underline{\underline{\Theta(n^2\log n)}}
\end{aligned}$$

c) $T(n)=4T(n/2)+n^3$

$$\begin{aligned}
T(n) &= 4T(\tfrac{n}{2})+n^3 \quad ;\; a=4,\; b=2,\; f(n)=n^3 \\
n^{\log_b a} &= n^{\log_2 4} = n^2 \\
f(n) &> n^{\log_b a} \rightarrow Fall\,3 \\
\rightarrow T(n) &= \Theta(f(n)) = \underline{\underline{\Theta(n^3)}}
\end{aligned}$$

16. Nutzen Sie einen generierten Lorem-Ipsum-Text über 50 Wörter und wenden Sie eine Schlüsselgeneratorfunktion Ihrer Wahl auf die Wörter an. Beschreiben Sie das Schlüsseluniversum und geben Sie die Menge an verwendeten Schlüsseln an. Tipp: Ein programmierbares Tabellenkalkulationsprogramm ist hierbei eine große Hilfe.

???

17. Verwenden Sie je einmal die modulare Hash-Funktion und einmal die Multiplikationsmethode, um die Schlüssel der vorhergehenden Aufgabe auf eine Hash-Tabelle mit Verkettungsmöglichkeit und 16 Buckets zu verteilen. Achten Sie dabei darauf, dass jedes Wort nur einmal je Hash-Funktion betrachtet wird!

???

18. Bestimmen Sie anhand der Ergebnisse der vorhergehenden Aufgabe, welche der beiden Hash-Funktionen das einfach uniforme Hashing eher erfüllt.

???

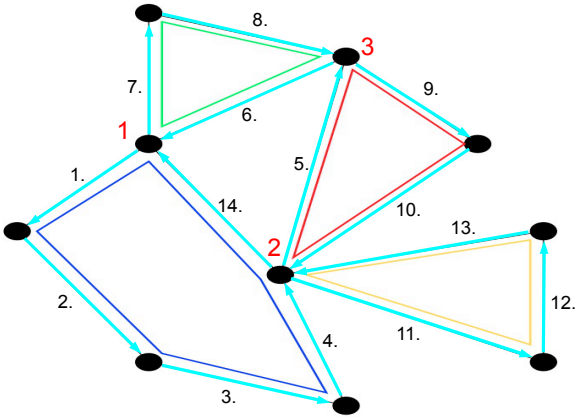
19. Ist jeder zyklenfreie, ungerichtete Graph ein Wald? Begründen Sie!

Nein, ein Wald besteht aus mehreren Bäumen.

20. Zeigen Sie, dass in jedem Graphen G = (V, E) mit m Kanten $\sum_{v \in V} deg(v) = 2\mathrm{m}$ gilt.

Die Summe $\sum_{v \in V} deg(v)$ zählt die Endknoten aller Kanten des Graphen. Damit wird jede Kante zweimal gezählt.

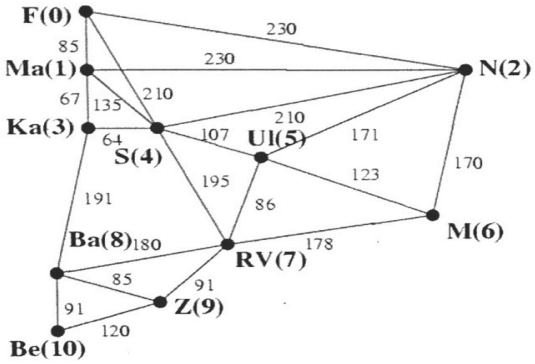
21. Konstruieren Sie für den Graphen aus dem Beweis von Satz 4.1 einen Eulerkreis, indem Sie mit anderen Zyklen beginnen.



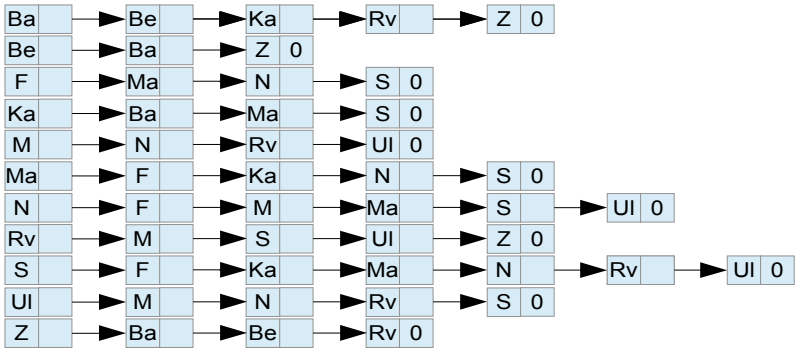
In diesem Beispiel starten wir in **Punkt 1** (beliebig). Von da aus gehen wir den blauen Kreis (beliebig) gegen den Uhrzeigersinn (beliebig) entlang (entsprechend meiner Nummerierung). Wir folgen diesem Kreis bis zum **Punkt 2**. Da wir dort ein (mind.) einem Kreis angekommen sind, in dem wir noch nicht waren, müssen wir in einen der beiden wechseln (aber egal welchen und egal wie rum). Wir entscheiden uns für den roten Kreis im Uhrzeigersinn. Am **Punkt 3** kommen wir wieder zu einem Kreis der noch nicht benutzt wurde und müssen in diesen wechseln. Wir entscheiden uns hier im Uhrzeigersinn weiter zu gehen. Am **Punkt 1** angekommen, dürfen wir diesmal nicht in den blauen Kreis wechseln, das wir einen Kreis nur an dem Punkt wieder betreten dürfen, an dem wir ihn verlassen haben, also bleiben wir im grünen Kreis. Am **Punkt 3** angekommen wechseln wir wieder zurück in den roten Kreis (es ist die einzige Möglichkeit und wir haben den Kreis dort verlassen). Zurück an **Punkt 2** haben wir nun die Möglichkeit in den gelben oder blauen Kreis zu wechseln. In den blauen Kreis dürfen wir an dieser Stelle zwar schon (wir haben den blauen Kreis hier verlassen), da der gelbe Kreis aber noch nicht betreten wurde, müssen wir diesen vorziehen. Einmal entgegen dem Uhrzeigersinn um den gelben Kreis herum gegangen kommen wir wieder zurück zu **Punkt 2** von wo aus wir dann in den blauen Kreis und zurück zu **Punkt 1** gelangen.

22. Geben Sie für den bewerteten Süddeutschlandgraphen die Adjazenzlisten und die Liste von Adjazenzlisten an.

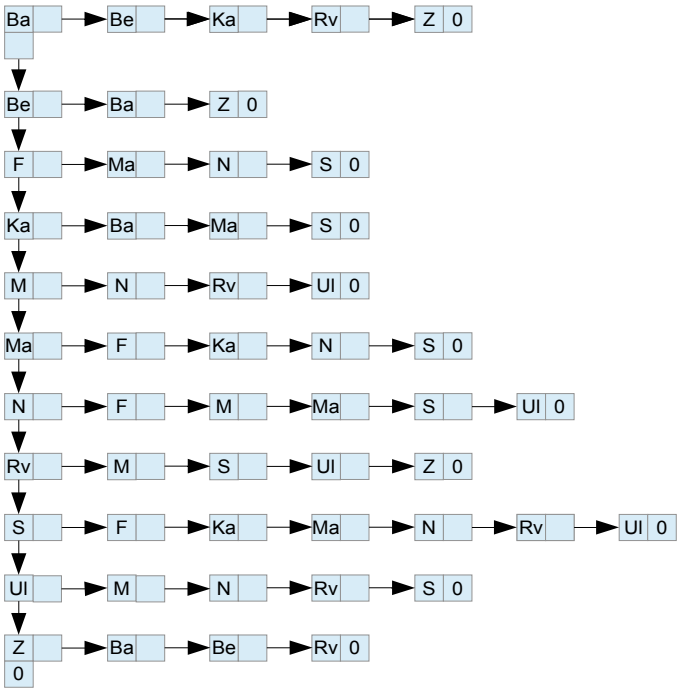
Süddeutschlandgraph:



Adjazenzliste:



Liste von Adjazenzlisten:

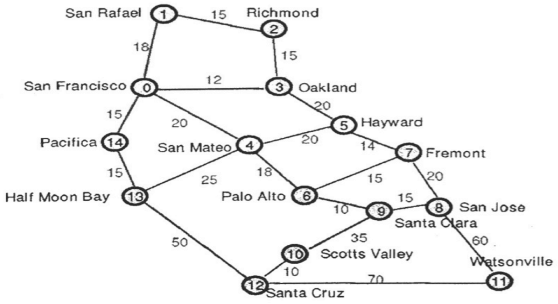


Anmerkung: Das leere Kästchen welches vom ersten Kästchen (Ba) aus nach unten wegführt steht dort exemplarisch für alle, d.h. eigentlich müsste das überalle noch sein.

23. Geben Sie für den Graphen aus Beispiel 4.2 die Adjazenzmatrix an.

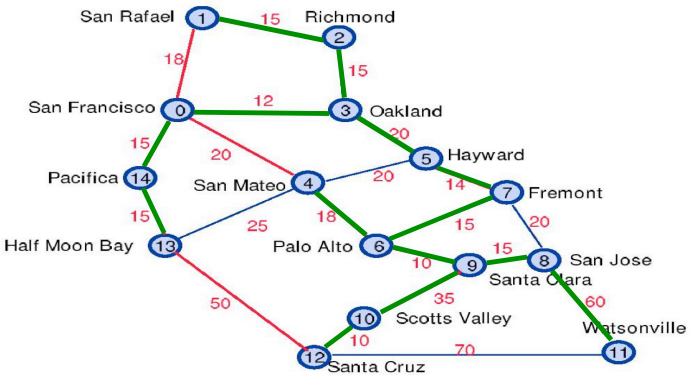
		VON							
		A	B	C	D	E	F	G	H
NACH	A			X	X				
	B	X							
	C		X						
	D	X							
	E	X	X						
	F						X		
	G						X		
	H								

24. Erzeugen Sie einen minimal aufspannenden Baum mit dem Algorithmus von Kruskal für den SFBay-Graphen:



Zuerst sortieren wir die Stecken der länge nach aufsteigend:

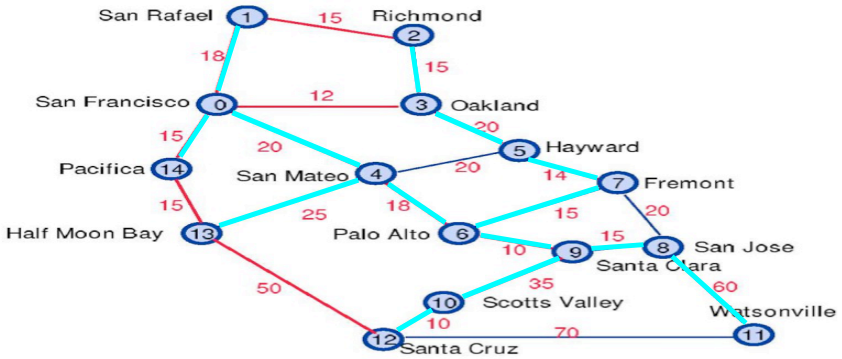
$$
\begin{aligned}
|(12,10)| &= 10 \\
|(9,6)| &= 10 \\
|(3,0)| &= 12 \\
|(7,5)| &= 14 \\
|(7,6)| &= 15 \\
|(2,1)| &= 15 \\
|(3,2)| &= 15 \\
|(9,8)| &= 15 \\
|(14,0)| &= 15 \\
|(14,13)| &= 15 \\
|(1,0)| &= 18 \\
|(6,4)| &= 18 \\
|(4,0)| &= 20 \\
|(5,3)| &= 20 \\
|(5,4)| &= 20 \\
|(8,7)| &= 20 \\
|(13,4)| &= 25 \\
|(10,9)| &= 35 \\
|(13,12)| &= 50 \\
|(11,8)| &= 60 \\
|(12,11)| &= 70
\end{aligned}
$$



Wir markieren vom kürzestens ausgehend alle Strecken wenn dadurch kein Kreis eintsteht, solange bis alle Knoten verbunden sind.

25. Verwenden Sie den Algorithmus von Dijkstra für das Single-Source-Shortest-Path-Problem beim SFBay-Graphen mit Start in Palo Alto.

/* Verzeiht mir, aber bei dieser Aufgabe war ich ehrlich gesagt zu faul eine Step-by-Step Anleitung zu schreiben. Das Prinzip ist aber auch in der ein oder anderen Klausuren nachlesbar. */



S	**{6}**
δ(S)	{4 [18], 7 [15], 9 [10]}
S	{6, 9}
F	{(6, 9)}
δ(S)	{4 [18], 7 [15], 8 [25], 10 [45]}
S	{6, 7, 9}
F	{(6, 7), (6, 9)}
δ(S)	{4 [18], 5 [29], 8 [25], 10 [45]}
S	{4, 6, 7, 9}
F	{(6, 4), (6, 7), (6, 9)}
δ(S)	{0 [38], 5 [29], 8 [25], 10 [45], 13 [43]}
S	{4, 6, 7, 8, 9}
F	{(6, 4), (6, 7), (6, 9), (9, 8)}
δ(S)	{0 [38], 5 [29], 10 [45], 11 [85], 13 [43]}
S	{4, 5, 6, 7, 8, 9}
F	{(6, 4), (6, 7), (6, 9), (7, 5), (9, 8)}

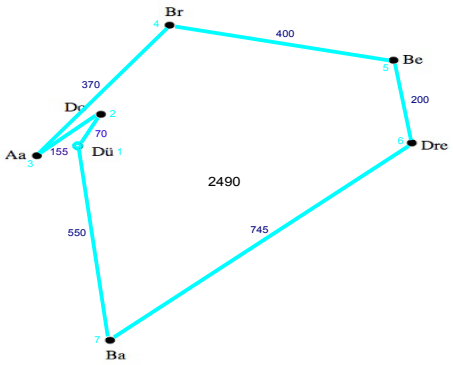
δ(S)	{0 [38], 3 [49], 10 [45], 11 [85], 13 [43]}
S	{0, 4, 5, 6, 7, 8, 9}
F	{(4, 0), (6, 4), (6, 7), (6, 9), (7, 5), (9, 8)}
δ(S)	{1 [56], 3 [49], 10 [45], 11 [85], 13 [43], 14 [53]}
S	{0, 4, 5, 6, 7, 8, 9, 13}
F	{(4, 0), (4, 13), (6, 4), (6, 7), (6, 9), (7, 5), (9, 8)}
δ(S)	{1 [56], 3 [49], 10 [45], 11 [85], 12 [93], 14 [53]}
S	{0, 4, 5, 6, 7, 8, 9, 10, 13}
F	{(4, 0), (4, 13), (6, 4), (6, 7), (6, 9), (7, 5), (9, 8), (9, 10)}
δ(S)	{1 [56], 3 [49], 11 [85], 12 [55], 14 [53]}
S	{0, 3, 4, 5, 6, 7, 8, 9, 10, 13}
F	{(4, 0), (4, 13), (5, 3), (6, 4), (6, 7), (6, 9), (7, 5), (9, 8), (9, 10)}
δ(S)	{1 [56], 2 [64], 11 [85], 12 [55], 14 [53]}
S	{0, 3, 4, 5, 6, 7, 8, 9, 10, 13, 14}
F	{(4, 0), (0, 14), (4, 13), (5, 3), (6, 4), (6, 7), (6, 9), (7, 5), (9, 8), (9, 10)}
δ(S)	{1 [56], 2 [64], 11 [85], 12 [55]}
S	{0, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14}
F	{(4, 0), (0, 14), (4, 13), (5, 3), (6, 4), (6, 7), (6, 9), (7, 5), (9, 8), (9, 10), (10, 12)}
δ(S)	{1 [56], 2 [64], 11 [85]}
S	{0, 1, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14}
F	{(0, 1), (0, 14), (4, 0), (4, 13), (5, 3), (6, 4), (6, 7), (6, 9), (7, 5), (9, 8), (9, 10), (10, 12)}
δ(S)	{2 [64], 11 [85]}
S	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14}
F	{(0, 1), (0, 14), (3, 2), (4, 0), (4, 13), (5, 3), (6, 4), (6, 7), (6, 9), (7, 5), (9, 8), (9, 10), (10, 12)}
δ(S)	{11 [85]}
S	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14}
F	{(0, 1), (0, 14), (3, 2), (4, 0), (4, 13), (5, 3), (6, 4), (6, 7), (6, 9), (7, 5), (8, 11), (9, 8), (9, 10), (10, 12)}
δ(S)	{}

26. Gegeben ist folgende Entfernungstabelle deutscher Städte:

Aachen						
Basel	545					
Berlin	650	875				
Bremen	370	775	400			
Dortmund	155	555	495	235		
Dresden	645	745	200	490	515	
Düsseldorf	90	550	560	285	70	580
	Aachen	Basel	Berlin	Bremen	Dortmund	Dresden

a) Bestimmen Sie mit dem Nearest-Neighbour-Algorithmus eine (heuristische) Lösung für das TSP-Problem. Starten Sie in Düsseldorf. Geben Sie die Tour sowie deren Länge an.

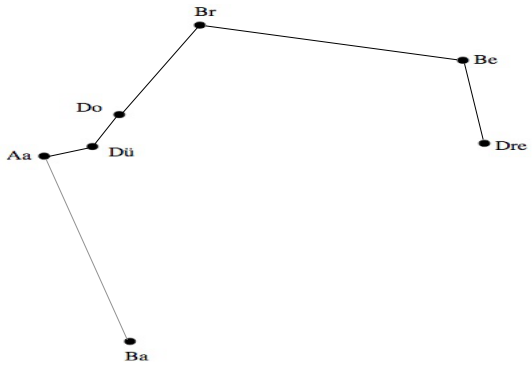
Der zugehörige, aber nicht gefragte Graph:



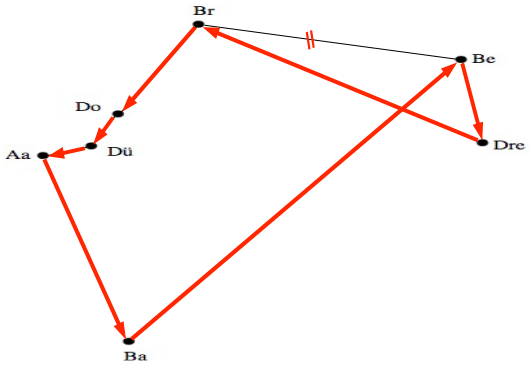
Von Dü ausgehend ist die kürzeste Strecken nach Do (70). Dann von Do ausgehend (ohne voriges zu beachten) ist die kürzeste Strecke nach Aa (155). Nun von Aa ausgehen (ohne voriges) geht's weiter nach Br (370). Von Br nach Be (400). Von Be nach Dre (200). Von Dre nach Ba (745) und dann zurück nach Dü (550). Das ergibt eine Gesamtstrecke von 2490.

b) Bestimmen Sie einen Minimum Spanning Tree mit dem Kruskal-Algorithmus. Zeichnen Sie diesen in die Landkarte ein.

Wir sortieren die Strecken wieder der länge nach aufsteigend und markieren dann, beim kürzesten beginnend, alle Strecken der Reihe nach, bis alle Knoten verbunden sind. Wir markieren **nicht** die Strecken die einen Kreis bilden würden.



c) Verwenden Sie die Minimum-Spanning-Tree-Heuristik mit Wurzelknoten Bremen zur Bestimmung einer Lösung des TSP-Problems. Zeichnen Sie die Lösung in die Karte ein und geben Sie die Tour sowie die Länge an.



Beginnend bei **Bremen** gehen wir in unserem Beispiel nach **Dortmund** (könnten aber auch erst nach Berlin wenn wir wollten), dann folgen wir dem Minimum-Spanning-Tree bis nach **Basel**, von wo aus es nicht weiter geht. Wir gehen dann gedanklich zurück bis wir den ersten Knoten treffen von dem aus es eine Strecke zu einem noch nicht verbundenen Knoten gibt. In diesem Fall ist dieser **Berlin**, wir merken uns diesen Knoten und löschen die Strecke, gehen wieder zurück nach **Basel** und verbinden diese beiden nun. Wir gehen wie weiter dem Minimum-Spanning-Tree entlang nach **Dresden** und von dort aus direkt zurück nach **Bremen**.

27. Versuchen Sie, die Minimum-Spanning-Tree-Heuristik zur Lösung des TSP-Problems beim SFBay-Graphen anzuwenden. Vergleichen Sie das Ergebnis mit dem Greedy-Algorithmus bei Start in San Franzisko oder anderen Städten. Warum treten hier Probleme auf?

???

28. Bestimmen Sie eine optimale Tour für das TSP-Problem beim SFBay-Graphen.

???

29. Für einen planaren (ebenen) ungerichteten Graphen sei A die Menge der durch die Kanten des Graphen begrenzten Flächenstücke. Überprüfen sie an allen bisher verwendeten Graphen (V, E) die Gültigkeit der Euler-Formel

$$|V|-|E|+|A|=2.$$

Graph Aufgabe 21:

???

30. Es soll ein Programm entworfen werden, das bei gegebener Abstandsmatrix für vollständig vernetzte Graphen eine optimale Tour findet.

a) Schreiben Sie ein Programm für OPTIMAL_TSP mit 4 Städten.

???

b) Schreiben Sie ein Programm für OPTIMAL_TSP mit n Städten.

???

31.

a) Geben Sie eine Grammatik an, die alle Zeichenketten der Form
***ab, abab, ababab, …* erzeugt.**

$$G = (\{A, B\}, \Sigma, P, A)$$
$$\Sigma = \{a, b\}$$
$$P = \{A \to aB, B \to bA | b\}$$

b) Geben sie einen regulären Ausdruck an, der alle Zeichenketten der Form
***ab, abab, ababab, …* erzeugt.**

(ab)+

32. Geben Sie eine Grammatik an für die Sprache $L = \{a^i b^j c^k | i \in \mathbb{N}, j \in \mathbb{N}, k \in \mathbb{N}\}$.

$$G = (\{A, B, C\}, \Sigma, P, A)$$
$$\Sigma = \{a, b, c\}$$
$$P = \{A \to aA | aB, B \to bB | bC, C \to cC | c\}$$

33. Geben Sie reguläre Ausdrücke für…

a) …groß geschriebene Worte wie z.B. Hallo, aber nicht HALLO.

$[A-Z][a-z]+$

b) …groß geschriebene Worte mit mindestens 3 und höchstens 5 Buchstaben.

$[A-Z][a-z]\{2,4\}$

c) …Gleitpunktzahlen mit beliebig vielen Stellen vor dem Komma und mindestenseiner Stelle nach dem Komma.

$-?[0-9]+\backslash.[0-9]+$

34. Geben Sie für jede in Aufgabe 33 angegebenen Sprachen einen endlichen Automaten an, der diese erkennt.

a)

$$\begin{aligned} M &= (\{K,G\},\{a,\ldots,Z\},\delta,G,\{K\}) \quad mit \\ \delta &= \{G,A\ldots Z \rightarrow K;K,a\ldots z \rightarrow K\} \end{aligned}$$

b)

$$M = (\{K_1,K_2,K_3,K_4,G\},\{a,\ldots,Z\},\delta,G,\{K_3,K_4,K_5\}) \quad für$$

$$\delta = \begin{Bmatrix} G, & A\ldots Z & \rightarrow & K_1; \\ K_1, & a\ldots z & \rightarrow & K_2; \\ K_2, & a\ldots z & \rightarrow & K_3; \\ K_3, & a\ldots z & \rightarrow & K_4; \\ K_4, & a\ldots z & \rightarrow & K_5 \end{Bmatrix}$$

c)

$$M = (\{Z_1,Z_2,Z_3,Z_4\},\{0,\ldots,9,.\},\delta,Z_1,\{Z_4\}) \quad für$$

$$\delta = \begin{Bmatrix} Z_1, & 0\ldots 9 & \rightarrow & Z_1; \\ Z_1, & 0\ldots 9 & \rightarrow & Z_2; \\ Z_2, & . & \rightarrow & Z_3; \\ Z_3, & 0\ldots 9 & \rightarrow & Z_4; \\ Z_4, & 0\ldots 9 & \rightarrow & Z_4 \end{Bmatrix}$$

35. Beschreiben Sie die durch folgende regulären Ausdrücke definierten Sprachen und geben Sie Beispiele an.

a) \\ (index | color | label | ref) \{ [^\}]* \}

Erst ein ****, dann entweder **index**, **color**, **label**, oder **ref**, dann ein **{** gefolgt von einer beliebigen Anzahl an Symbolen ohne das **}**, und dann zum Schluss **}**.
Bsp.: \index{1wedqw} \color{asd} \label{2d.} \ref{u}

b) % .*

Erst ein **%**, dann eine beliebige Anzahl beliebiger Zeichen.
Bsp.: %hallo %bye

c) \\ section *? \{ .* \}

Erst ein ****, dann **section**, dann null oder ein *****, dann ein **{**, eine beliebig lange Folge beliebiger Zeichen und zum Schluss noch ein **}**.
Bsp.: \section{hallo} \section*{bye}

d) [A-Za-z0-9]+ @ [A-Za-z0-9]+ (\. [A-Za-z0-9]+){1,6}

Mindestens ein Zeichen von **0-9** oder von **a-Z**, gefolgt von einem **@**, danach wieder mindestent ein Zeichen von **0-9** oder von **a-Z**, gefolgt von einem **.** und dann noch mindestens ein und höchstens 6 Zeichen von **0-9** oder von **a-Z**
Bsp.: emrichm@weingarten.de

36. Konstruieren Sie (deterministische oder nichtdeterministische) endliche Automaten für folgende durch reguläre Ausdrücke gegebenen Sprachen:

a) [0-9]*\.[0-9]+

$$M = (\{Z_0, Z_1, Z_2\}, \{0, \ldots, 9, .\}, \delta, Z_0, \{Z_2\}) \quad mit$$

$$\delta = \begin{pmatrix} Z_0, & 0 \ldots 9 & \rightarrow & Z_0; \\ Z_0, & . & \rightarrow & Z_1; \\ Z_1, & 0 \ldots 9 & \rightarrow & Z_2; \\ Z_2, & 0 \ldots 9 & \rightarrow & Z_2 \end{pmatrix}$$

b) \\section*?\{.*\}

$$M = (\{Z_0, \ldots, Z_{11}\}, \{alle\}, \delta, Z_0, \{Z_{11}\}) \quad mit$$

$$\delta = \begin{pmatrix} Z_0, & \backslash & \rightarrow & Z_1; \\ Z_1, & s & \rightarrow & Z_2; \\ Z_2, & e & \rightarrow & Z_3; \\ Z_3, & c & \rightarrow & Z_4; \\ Z_4, & t & \rightarrow & Z_5; \\ Z_5, & i & \rightarrow & Z_6; \\ Z_6, & o & \rightarrow & Z_7; \\ Z_7, & n & \rightarrow & Z_8; \\ Z_7, & n & \rightarrow & Z_9; \\ Z_8, & * & \rightarrow & Z_9; \\ Z_9, & \{ & \rightarrow & Z_{10}; \\ Z_{10}, & 0 \ldots Z & \rightarrow & Z_{10}; \\ Z_{10}, & \} & \rightarrow & Z_{11}; \end{pmatrix}$$

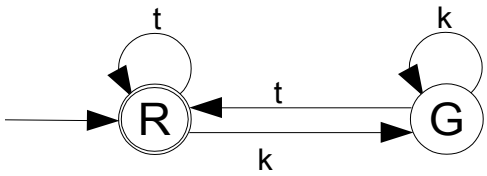
37. Es soll eine Fußgängerampel mit Hilfe eines endlichen Automaten programmiert werden. Die Ampel hat zwei Zustände rot und grün (aus der Sicht des Fahrzeuges). Im Zustand rot akzeptiert die Ampel Signale von der Kontaktschleife auf der Straße und schaltet dann auf Grün. Im Zustand Grün akzeptiert die Ampel Signale vom Fußgängertaster und schaltet auf Rot. Alle anderen Eingaben ignoriert der Automat.

a) Geben Sie einen endlichen Automaten für diese Aufgabe an.

$$M = (\{R, G\}, \{k, t\}, \delta, R, \{R\})$$

$$\delta = \begin{pmatrix} R, & k & \rightarrow & G; \\ R, & t & \rightarrow & R; \\ G, & t & \rightarrow & R; \\ G, & k & \rightarrow & G \end{pmatrix}$$

b) Zeichnen Sie ein Zustandsdiagramm zu diesem Automaten.


t
k
t
R
G
k

38. Es soll ein Getränkeautomat mit Hilfe eines endlichen Automaten programmiert werden. Der Automat kann mit bis zu 4 Dosen Mineralwasser, 4 Dosen Limo und 4 Dosen Bier gefüllt werden. Wenn eine 1-Euro-Münze eingegeben wird, soll er eine Dose des gewählten Getränks ausgeben. Bei Eingabe einer anderen Münze soll er die eingegebene Münze wieder ausgeben, aber kein Getränk. Wenn von einer Getränkesorte alle Dosen ausgegeben sind, soll er anhalten und per Funk den Service benachrichtigen.

a) Geben Sie einen endlichen Automaten (mit Ausgabe) für diese Aufgabe an.

???

b) Zeichnen Sie ein Zustandsdiagramm zu diesem Automaten.

???

c) Geben Sie einen regulären Ausdruck für diese Sprache an.

???

d) Geben Sie eine reguläre Grammatik an, welche die für diesen Automaten erlaubte Eingabesprache akzeptiert.

???

39. Kontruieren Sie einen deterministischen endlichen Automaten, der zu dem aus Beispiel 5.15 äquivalent ist.

$$M = (\{S_0, S_1, T, E\}, \{a, b, \epsilon\}, \delta, S_0, \{E\})$$

$$\delta = \{S_0; a \rightarrow S_1, S_1; a \rightarrow S_1, S_1, b \rightarrow T; T, b \rightarrow T; T, \epsilon \rightarrow E\}$$